WorldEpisode: Auditing Silent State Drift in Robot-Learning Datasets

Alba Tellez Fernandez
URDF Studio

Abstract

Interactive spatial datasets can be locally valid and behaviorally wrong. A robot log can load while commands are replayed at command time instead of motor-effective time; a benchmark can report strong policy performance while the same reconstructed scene appears in both train and test; a simulator asset can render while its collision role, coordinate frame, or digest no longer matches the state under evaluation. We call this failure mode *silent state drift*: the bytes are readable, but the state assumptions that make replay, training, or deployment meaningful have drifted.

This paper presents *WorldEpisode*, an executable state-integrity contract for robot-learning datasets. Rather than adding another file format, it specifies the invariants an episode must preserve on top of existing containers such as LeRobotDataset, Rerun, MCAP, NCore, OpenUSD, and glTF: persistent identity, explicit frame and clock graphs, state and asset roles, transition and action semantics, temporal events, provenance, quality records, content-addressed assets, and lineage-aware splits (we call this general family of invariants Universal Spatial State, or USS, but evaluate only the robotics profile in this paper).

We evaluate WorldEpisode on public robot-learning data. On ArmnetBench's public SO-101 LeRobot release, a random episode split leaks every tested world lineage and yields 0.850 offline success for a behavioral-cloning probe; a scene-disjoint split reduces measured leakage to zero (0.000) and the same probe to 0.000. On a real SO-101 trajectory, declaring the inferred four-frame, 133 ms actuation delay reduces validation joint RMSE from 4.732 to 1.862 degrees, and the same timestamp-aware action contract reduces same-trace MuJoCo and Genesis replay RMSE from 3.425 to 1.563 degrees. A programmatic LeRobotDataset-to-WorldEpisode-to-LeRobotDataset round trip over ten public episodes preserves 1,935 state/action rows, timestamps, video timestamp ranges, and physical-frame records with maximum numerical error 0.0 while reporting source-absent semantics as explicit conversion loss. The validator detects all 14 injected physical-coherence fault classes. Deterministic game-engine and autonomous-driving pilots only sketch how the same vocabulary might extend beyond robotics; they are illustrative, not measured evidence.

The contribution is a tested state-integrity layer and audit protocol for deciding whether spatial data can be trusted for conversion, replay, evaluation, and deployment. The paper does not claim a new universal file format, simulator-independent physics, or benchmark score inflation without policy reruns. ACT/Diffusion rollouts, famous-benchmark reruns, contact-rich simulator comparisons, Isaac/SAPIEN execution, external adoption, and production game or autonomous-driving audits remain explicit open gates.

1 Introduction

Summary

The problem. Interactive spatial datasets can pass every local loader and still be behaviorally wrong: joint values in an undeclared unit, command timestamps replayed as motor-effective time, train and test sharing the same reconstructed scene, a dropped collision role, or an undeclared clock offset. We call this *silent state drift* — the bytes are readable, but the state assumptions that make replay, training, or deployment meaningful have drifted.

Why it goes uncaught. LeRobotDataset, Rerun, MCAP, NCore, OpenUSD, and glTF each store a piece, but none bind an episode to an immutable world revision with persistent identity, frame and clock mappings, action-timing, representation roles, and provenance. The file validates; the contract does not exist.

Why it matters. The result is weeks of wasted training, benchmark scores inflated by scene leakage, and real-to-sim gaps blamed on the simulator rather than the data contract.

What this paper contributes. *WorldEpisode*, an executable sidecar contract of those invariants for robot-learning datasets: a validator and a one-line preflight that turn silent failures into hard, auditable errors before training or deployment.

Imagine training a behavioral-cloning policy for three weeks on a public robot dataset. The loss curves are smooth, the simulator reports an 85% success rate, and every Parquet, video, and scene file passes its local loader. Then the policy is deployed on the real arm and immediately misses the object. The failure is not a new neural architecture problem. The dataset silently encoded joint values in a unit the controller did not declare, the replay script treated command timestamps as motor-effective timestamps, and the random split let the policy see the same reconstructed room in both training and evaluation. Nothing was corrupt, but the experiment was scientifically broken. We call this failure class *silent state drift*. The same pattern of a file that validates while its behavior is wrong appears outside robotics as well; Section 7 returns to that point as future work, since this paper's measured evidence is entirely robotics.

Table 1 makes the stakes concrete before any evaluation: each row is a file that still loads, the behavior it silently breaks, and the requirement that catches it. The rest of the paper turns each requirement into an executable rule and measures what happens when it is missing.

Robotics is the profile implemented and evaluated here because it is the deepest current stress test: it combines real hardware, asynchronous control, multi-camera data, reconstructed worlds, simulator exports, and policy evaluation.

Existing formats solve important pieces of this chain. The LeRobot library vertically integrates robot learning from motor interfaces through dataset collection, storage, streaming, policies, and deployment [3]. The current LeRobotDataset v3 layout stores high-frequency states, actions, and timestamps in Parquet, visual streams in MP4 shards, and episode indexing in metadata rather than filenames [8]. Rerun provides a physical-data layer and storage target for multimodal recordings. NCore documents explicit pose graphs and sensor conventions. OpenUSD, SimReady, glTF, and simulator-native formats describe composed worlds and assets. Gaussian splats are also moving into standard asset containers through glTF's `KHR_gaussian_splatting` extension [20, 21]. Recent real-to-sim systems such as RoboSnap and DROID-Sim show that Gaussian-splat environments can support trajectory replay, synthetic data generation, and policy evaluation at dataset scale [19]. That progress makes the missing contract more urgent rather than less. A layered real-to-sim scene can be visually faithful while still dropping the collision role of an object, and a replayable trajectory can still be wrong if the policy action vector is interpreted under a different controller contract at deployment time.

Table 1: Failure modes that pass local validation but break behavior, and the WorldEpisode requirement that catches each. Requirement IDs are defined in Section 2.

File still loads, but...	Silent consequence	Requirement
joint values use an undeclared unit	controller misreads the scale; policy misses the object	`ACTION.001`
command time is replayed as motor-effective time	replay and deployment drift off the real trajectory	`ACTION.002`
a random split shares a reconstructed scene across train and test	success is inflated and does not transfer	`SPLIT.001`
a real-to-sim export keeps appearance but drops the collision proxy	the policy collides with geometry it cannot see	`REP.001`
camera and lidar logs are fused across clock domains	spatial fusion error beyond tolerance	`TIME.002`
an asset URI looks right but its content digest changed	the wrong geometry is used under evaluation	`ASSET.002`

WorldEpisode is:

- **storage-neutral**: serializable through LeRobotDataset v3, Rerun, NCore, MCAP, game telemetry, AV logs, or a reference package;
- **representation-neutral**: compatible with splats, meshes, NeRFs, point clouds, collision proxies, sensor streams, and future assets;
- **runtime-neutral**: mappable to MuJoCo, Isaac Sim, SAPIEN, Genesis, game engines, AV replay systems, and future runtimes;
- **loss-explicit**: conversions may be lossy, but never silently lossy.

The empirical claims in this release are intentionally bounded: they demonstrate executable failure mechanisms and auditable conversion semantics, not a completed multi-lab benchmark, production game engine audit, autonomous-driving fleet audit, or real-robot deployment study.

Contributions.

1. **WorldEpisode**, an executable state-integrity contract for robot-learning datasets, expressed as five graphs of invariants (identity; frames and clocks; representation roles; temporal state and events; provenance and lineage), with a JSON Schema, a semantic validator, and a one-line fail-closed preflight for native LeRobot and Rerun artifacts.
2. **Loss-explicit binding**: round-trip conversions that emit machine-readable loss reports, with measured native-versus-sidecar semantic retention.
3. **Controlled failure studies**: scene-lineage leakage, action-timing replay drift, and real-to-sim contract drift, plus two non-robotics pilots — each an executable mechanism, with every stronger claim bounded by an explicit open reproduction gate.

Reader's guide. A reader who only wants the motivation can stop after Table 1. The rest follows one spine: Section 2 states the five questions any valid state must answer, Section 3 formalizes

them as five graphs, Section 4 delivers them as a sidecar contract, Section 5 makes the contract executable, Section 6 is the empirical core with each result tagged by the requirement it proves, and Section 7 states the boundary of every claim.

2 What a Valid Spatial State Must Answer

The spine starts here: five questions any interactive spatial state must answer before it is converted, replayed, or evaluated, plus one question of scale. Each maps to a stable requirement ID that Section 5 enforces and Section 6 tests.

Any valid spatial state must answer a practical question: what must be known before it can be safely converted, replayed, synchronized, augmented, or used for evaluation? A valid state record must answer five questions. A production dataset must also answer how those records are named, sharded, indexed, resolved, and versioned without turning storage layout into the semantic API. WorldEpisode specializes these requirements for robot-learning episodes.

Which state revision am I in? (`WORLD.001`, `TRACE.001`) Each record references one immutable, content-addressed state or world revision plus an ordered list of state-specific deltas. In robotics this is a base world plus episode deltas; in a game engine it may be a map patch plus live actor state; in an AV replay it may be a map/sensor-calibration revision plus log-time deltas. Changing geometry, metric scale, calibration, collision proxies, physical parameters, entity decomposition, object identity, or task-relevant fixtures creates a new revision. This makes it detectable when two datasets silently use the same filename for different states, or different filenames for the same state.

What entities does the state contain? (`ENTITY.001`) Objects, robot links, vehicles, sensors, avatars, fixtures, support surfaces, semantic regions, and generated assets must have persistent identifiers. A mug in a segmentation mask, a Gaussian group, a collision mesh, a simulator actor, a language annotation, and a contact event should resolve to the same entity when they refer to the same physical object. A game obstacle or AV tracked object needs the same identity discipline across telemetry, assets, and runtime actors.

Where and when is this state valid? (`FRAME.001`, `TIME.001`, `ACTION.002`) Spatial values declare their frames, units, transform direction, validity interval, and calibration revision. Streams declare clock domains, timestamps, synchronization mappings, and timing error where available. Actions declare command time and effective motor time rather than assuming those are the same instant. WorldEpisode calls these state transition invariants: a control input, physics step, server patch, or sensor fusion event is only meaningful under its declared timing, frame, unit, and latency contract.

What was preserved or lost during conversion? (`CONVERT.001`) WorldEpisode reuses existing standards rather than replacing them. USD [1] carries composed simulation worlds, glTF carries Gaussian appearance assets [20], NCore carries capture and pose-graph conventions [14], Rerun carries physical-data recordings with column chunks [18], ROS/MCAP carries logs, and domain-native engines carry their own telemetry. A converter may externalize or approximate fields, but it must emit a machine-readable loss report instead of silently discarding semantics.

Can the evaluation split leak? (`SPLIT.001`) Record-level random splits are not enough when multiple records share a reconstructed room, map patch, physical object, source capture, calibration, generated asset lineage, or state ancestry. A WorldEpisode split manifest can enforce disjointness by state lineage, physical entity, source capture, reconstruction run, generated asset family, or collection site.

Can this scale past one dataset folder? (`DATASET.001`) Large robot corpora require a dataset manifest above the episode records. The manifest declares namespaces, resolver priority, shard catalogs, materialized indexes, split catalogs, and append-only versions. This lets a reader locate all SO-101 pick-place traces for a world-disjoint split by index lookup rather than by crawling storage paths.

The initial WorldEpisode profile is intentionally narrow: rigid tabletop manipulation with fixed-base single- or dual-arm robots. That bounded domain keeps conformance executable rather than aspirational. Future profiles can extend this specification to humanoids, locomotion, deformables, fluids, multi-agent game worlds, and autonomous-driving logs.

3 The Five Invariants

The same five questions from Section 2, now formalized as five coupled graphs. Making them mutually referential is what turns a silent error into a detectable one.

Figure 1 shows the five coupled graphs that make a spatial state replayable and auditable rather than just stored. Each graph answers a question that appears during debugging: what object is this, where and when is it, what is the asset safe to do, what physically or virtually happened, and where did this data come from?

Takeaway. *Five graphs answer five debugging questions and are made mutually referential, so an identity, timing, role, event, or lineage error becomes detectable instead of hidden.*

3.1 Identity graph: what object is this? (`ENTITY.001`)

Every physical or logical entity receives a persistent identifier: robot, link, gripper, vehicle, avatar, sensor, rigid object, articulated object, fixture, support surface, region, human, light, or semantic group. The same `entity_id` must be used by segmentation masks, bounding boxes, Gaussian groups, render meshes, collision bodies, simulator actors, language references, contact events, object trajectories, grasp events, and attachment events. The real-world fix is simple: if a language command says "pick up the mug," the perception mask, the simulator object, and the collision proxy must identify the same mug.

3.2 Frame and clock graph: where is it, and exactly when? (`FRAME.001`, `TIME.001`)

Every spatial value declares source frame, target frame, units, handedness, up axis, transform direction, quaternion convention, validity interval, calibration revision, and uncertainty where available, consistent with and extending ROS's coordinate and unit conventions [15]. Every stream declares a clock domain, device timestamp, host-receive timestamp where available, exposure interval for cameras, effective command time for actions, synchronization mapping, and estimated offset/drift/error. NCore's explicit pose-graph convention demonstrates the kind of rigor WorldEpisode requires across the whole state record [14]. This graph is what turns a suspicious replay from "the policy is bad" or "the engine is wrong" into a diagnosable offset, transform direction, or unit error.

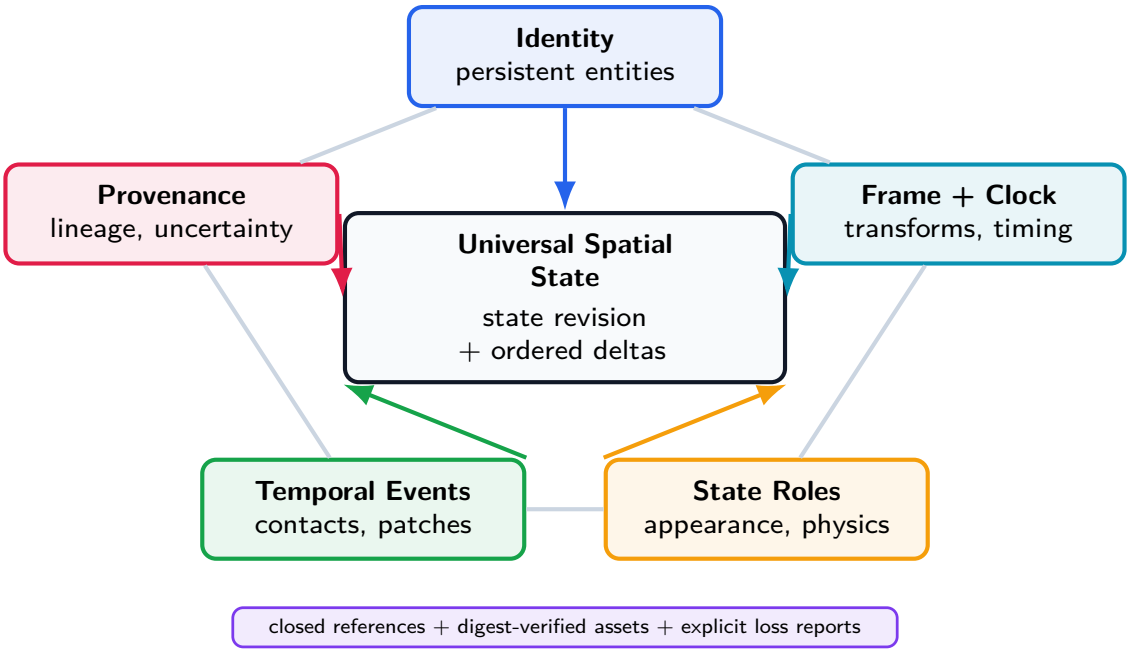

Figure 1: The five-graph model. WorldEpisode makes identity, frames/clocks, state roles, temporal events, and provenance mutually referential so conversion, synchronization, and replay errors become detectable instead of hidden.

3.3 Representation-role graph: what is this asset safe for? (`REP.001`)

One physical or virtual entity can have many representations: Gaussian splat appearance, textured render mesh, metric surface mesh, depth point cloud, collision proxy, mass/inertia record, semantic affordance, telemetry stream, or learned embedding. Each representation declares its role and an asset descriptor: URI, media type, SHA-256 digest, optional mirrors, optional embedded payload metadata, coordinate frame, units, validity interval, derivation, uncertainty, and license. The URI may be relative, HTTP(S), Hugging Face, object storage, OCI, IPFS, or another registered resolver. Portability comes from deterministic resolution and digest verification, not from requiring every asset to live in one folder. The role field is essential: a splat may be valid for appearance but invalid for collision, while a simplified proxy may be valid for collision but unsuitable for visual training.

3.4 Temporal-event graph: what actually happened? (`TRACE.001`)

Spatial records do not store only poses. They record interaction events: contact begin/end, grasp establish/release, attachment create/remove, support, containment, map or collision patch, articulation state changes, spawn/despawn, topology changes, simulator reset, intervention, subgoal completion, and failure detection. Each event references entities, time, confidence, source, and optionally evidence. This turns raw coordinate streams into milestones such as "contact established," "object dropped," "authoritative collision patch applied," or "intervention occurred."

3.5 Provenance graph: where did this come from? (`PROV.001`, `SPLIT.001`)

Every derived asset identifies source episodes, source sensors and intervals, reconstruction algorithm, software version, model/checkpoint hash, configuration, calibration revision, manual edits, source and output hashes, creator, license, and quality metrics. WorldEpisode maps to existing provenance

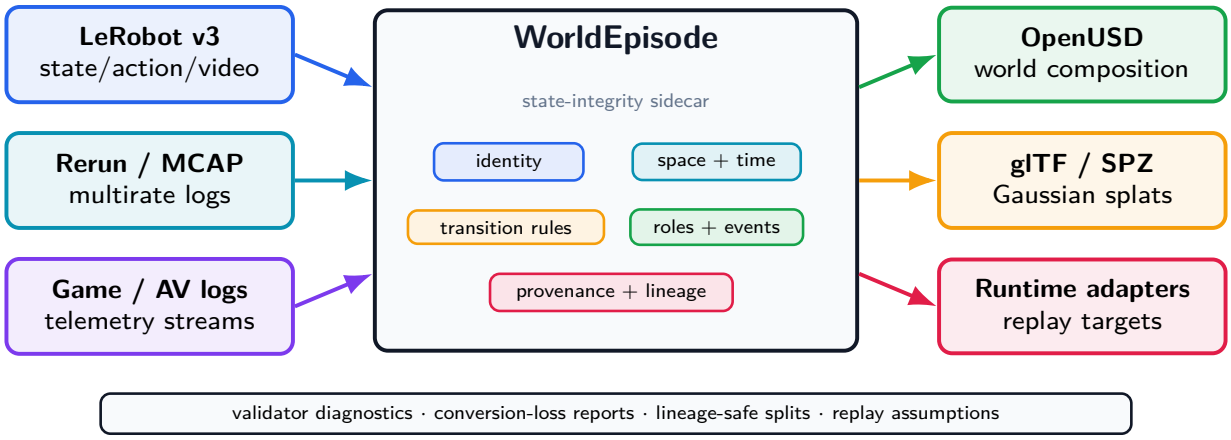

Figure 2: WorldEpisode is the semantic bridge between spatial data containers, state/world assets, and runtime targets.

and dataset metadata vocabularies rather than inventing an incompatible publication vocabulary. This graph lets a split avoid putting the same reconstructed room, capture session, or generated asset family on both sides of evaluation.

4 The Sidecar Contract

How the five invariants are delivered: a sidecar wrapped around existing formats, plus a runtime-neutral adapter contract, so no container has to become another.

WorldEpisode is implemented as a sidecar blueprint around existing formats. It does not ask a LeRobot dataset to become a USD scene, a game-engine telemetry log to become an AV replay format, or a USD scene to become a policy-training table. Instead, it records the relationships those containers need in order to agree about the same state: the state revision, persistent entities, space/time conventions, transition contract, asset roles, interaction events, provenance, and split lineage.

Takeaway. *WorldEpisode records the cross-container relationships that loaders otherwise leave implicit; it does not force any format to become another.*

The sidecar is not a mandate to package a global robot corpus as millions of small folders. The single-episode record is the semantic unit. Production datasets add a manifest layer: globally scoped namespaces, URI resolvers, shard catalogs, materialized indexes, split manifests, and append-only release snapshots. Opening such a corpus starts from the manifest and its indexes; it does not require recursively scanning an object store or loading every episode.

Figure 2 shows the intended data flow. Dataset, log, and telemetry containers plug in on the left; world assets and replay targets plug in on the right; the WorldEpisode sidecar holds the semantics that would otherwise be scattered across undocumented loader assumptions.

This architecture gives the paper its central rule: keep native containers for the data they are good at storing, but make the cross-container assumptions explicit, checkable, and loss-aware.

4.1 WorldEpisode as a Meta-Simulator Contract

WorldEpisode is not tied to a single rendering or physics engine. It operates as a meta-simulator contract: a runtime becomes a trusted target by implementing an adapter that preserves the same

semantic invariants and emits replay evidence. This keeps the integration direction clean. Datasets are not rewritten around simulator-native assumptions; simulators compile their scene loading, action interfaces, and replay reports against the WorldEpisode sidecar.

The adapter contract has three layers:

- **Invariant interface**: ingests immutable world revisions, frame and clock graphs, entity identity, representation roles, action channels, asset digests, provenance, and quality records.
- **Asynchronous schema extension**: lets an engine register cloth, deformable, fluid, articulation, material, or procedural-scene extensions without weakening the core episode record.
- **Deterministic replay accountability**: records runtime identity, version, solver, timestep, actuator parameters, initialization state, command/effective timing, latency, tolerance envelopes, and measured drift.

The current implementation reflects this boundary across two codebases. WorldEpisode has tested minimal MuJoCo [7] and Genesis replay adapters for the same LeRobot control-loop trace. URDF Studio, the companion simulator-transfer workbench, implements the same episode-backend idea for MuJoCo and Genesis: both backends pass a shared `SimBackend` conformance suite, and a one-episode carton-sorting scenario produces a MuJoCo–Genesis comparison report with matching task success and measured trajectory divergence. The Isaac mapping is emitted and deployment-guarded but untested, and SAPIEN remains guarded rather than replay-tested. The claim is therefore not simulator-independent physics. The claim is runtime-neutral accountability: every simulator reports against the same state contract, so cross-runtime drift becomes measurable rather than hidden inside loader conventions.

5 WorldEpisode: The Executable Profile

The contract as running code: complete action semantics, immutable world revisions, loss-aware conversion, and the JSON Schema, validator, one-line preflight, converters, and adapters that Section 6 exercises.

5.1 Complete Action Semantics

An action vector is not portable without its operational contract. Each action channel must declare actuator, control mode, parameterization, reference frame, units, absolute/delta/velocity semantics, limits, command rate, command timestamp, effective timestamp, latency model, interpolation, normalization, gripper semantics, and missing-value policy. A vector such as $[0.01, 0, 0, 0, 0, 0, 1]$ is otherwise ambiguous: it could be a tool-frame delta, base-frame delta, velocity command, normalized policy output, or delayed target increment.

5.2 Immutable World Revisions

Each episode references one base world revision plus ordered episode deltas. A new revision is required when world geometry, metric scale, calibration, collision proxies, physical parameters, entity decomposition, object identity, or task-relevant fixtures change. Content hashes allow two datasets to prove that they refer to the same world without relying on filenames.

5.3 Loss-Aware Conversion

WorldEpisode supports many bindings, but it does not promise impossible universal losslessness. Every converter emits a machine-readable report listing preserved, externalized, approximated,

discarded, and warning fields. The standard permits lossy conversion; it forbids silent lossy conversion.

5.4 Implementation Artifact

We implement the contract as four executable components: a JSON Schema for structural validity, a machine-readable conformance requirement file, an installable semantic validator, and a round-trip experiment runner. The validator maps each diagnostic to a stable requirement identifier such as `TIME.001`, `FRAME.002`, or `ASSET.002`. The experiment runner takes a WorldEpisode package, exports binding-native artifacts plus sidecars, reimports them, compares a versioned semantic projection, injects controlled faults, and writes reproducible result artifacts. The projection itself is published as `conformance/projections/uss-core-23.v0.json` and validated against `schemas/semantic-projection-v0.schema.json`. URDF Studio remains the initial implementation root, but the contract is represented independently in the public schema and validator.

The validator is packaged as `worldepisode`. A blocking preflight can be run from the shell with `worldepisode preflight <dataset-or-manifest>` or from a training script with a single Python call such as `preflight_lerobot(dataset.root).raise_if_failed()`. Native LeRobot folders and Rerun `.rrd` recordings are recognized directly; without a WorldEpisode manifest or sidecar, they fail closed on missing world revision, entity identity, representation role, action timing, frame/clock, split-lineage, and conversion-loss controls. This makes the semantic audit a pre-training check rather than a separate post-hoc documentation exercise.

The repository also includes an active LeRobotDataset v3 converter, a scene-leakage and split-manifest tool, a control-replay tool, a dependency-free replay-adapter conformance harness, a dataset-scale manifest audit and performance benchmark, a clean-room reader that does not import `worldepisode`, and a meta-simulator adapter report covering MuJoCo, Genesis, Isaac, and SAPIEN. Section 6 presents what each one measures.

6 Evidence: Failure Case Studies

The empirical core. Each case is a file that passes local validation, the requirement it violates, and what changes once the contract is enforced. Requirement IDs in each heading tie the result back to Section 2.

The experiments are written as failure investigations rather than as isolated unit tests. Each one starts from a failure mode that can make an embodied or virtual-agent result look better, cleaner, or more reproducible than it really is. All measured evidence here is robotics; two deterministic non-robotics pilots are deferred to the future-work discussion in Section 7, where they are framed as illustrative rather than measured.

Reading the open-result boxes. Amber boxes mark work that is executable but not counted as a paper result. Each box states the exact command path, required artifacts, and acceptance rule needed before the corresponding claim can move from "open" to "measured." These boxes are intentionally visible in the PDF so a reader can reproduce or close the missing evidence without reading the repository internals.

Case 1: scene-lineage leakage in evaluation splits (`SPLIT.001`). The most dangerous failure is not a crash; it is a benchmark that looks successful for the wrong reason. We audit ArmnetBench v0.1's single-arm SO-101 LeRobot release [2] by deriving eight WorldEpisode-style `world_lineage` hashes, one for each task-scene and camera-layout group. A standard random episode split trains

on 320 episodes, tests on 80, and leaks all test scene lineages into training. The same Torch MLP state-action behavioral-cloning probe then reaches 0.850 offline imitation success under a fixed normalized-RMSE tolerance of 0.25.

When the split is made scene-disjoint, the apparent success disappears. The held-out eye-drop scene lineages have zero overlap with training, the test set contains 100 episodes, and offline BC success drops to 0.000. Mean episode normalized RMSE rises from 0.183 to 0.363, a 1.98× increase. This is an offline action-imitation benchmark, not a physical rollout claim or a claim about state-of-the-art LeRobot policies. Its purpose is narrower and diagnostic: it exposes that a conventional split can reward memorizing a background world rather than learning a transferable manipulation behavior. The committed split manifest makes the same leakage audit repeatable with ACT, Diffusion Policy, or another LeRobot-native policy. HELP: C1

Takeaway. *The number moved with the split, not the policy: a random split scores 0.850 by memorizing the scene, and a lineage-disjoint split drops the same probe to 0.000.*

To verify that the split packages are executable beyond the original MLP probe, we also run a closed-form temporal ridge baseline over the committed compact LeRobot state/action packages. The baseline uses a three-frame state history and no video. It reaches 0.925 offline success on the random episode split and 0.420 on the scene-disjoint split, a 0.505 success-rate drop, while mean episode normalized RMSE rises from 0.157 to 0.255. This is still not ACT or Diffusion and not a rollout result, but it shows that the leakage penalty survives a second temporal policy-style baseline executed directly on the packaged LeRobot splits.

To keep that stronger claim from being implied before it is measured, the repository also emits an ACT/Diffusion gate from the same split manifest. The gate writes train/test episode allowlists, four virtual split manifests with 27 digest-verified source files, four compact physical LeRobot state/action split packages totaling 241,470 rows, four LeRobot-native training jobs (`act` and `diffusion` on random and scene-disjoint splits), required offline action-evaluation reports, and a high-fidelity simulator or physical rollout contract. The gate is intentionally open in this release; no ACT, Diffusion Policy, IsaacLab, or hardware success number is claimed, and the compact packages omit video payloads until source videos are mirrored with committed digests.

Open result, not claimed: ACT/Diffusion and rollout impact

Status: open. To close this gate, first regenerate the split packages and LeRobot job specs with `python3 tools/lerobot_policy_leakage_gate.py`. Then run the generated jobs with `bash docs/experiments/lerobot_policy_gate/run_lerobot_policy_jobs.sh` inside a LeRobot training environment.

Required artifacts: policy checkpoint digests, train/eval configs and seeds, offline action metrics for random and scene-disjoint splits, rollout traces or videos with content digests, and an updated `docs/experiments/lerobot_policy_gate/policy_gate_report.json`.

Acceptance rule: at least one ACT or Diffusion Policy run must report both split protocols, and at least one simulator or physical rollout report must use the same split manifest. Until then, the paper claims only the offline Torch probe and the prepared gate.

Figure 3 summarizes the measured changes for three experiment families where explicit state semantics alter the outcome.

Case 2: loss-explicit dataset conversion (`CONVERT.001`). To test an actual LeRobotDataset rather than only a capability model, we implement LeRobotDataset v3 → WorldEpisode → LeRobotDataset v3 on five-episode batches from two pinned public datasets: `lerobot/svla_so101_pickplace` [11] and `lerobot/pusht` [10]. The converter downloads the real Parquet meta-

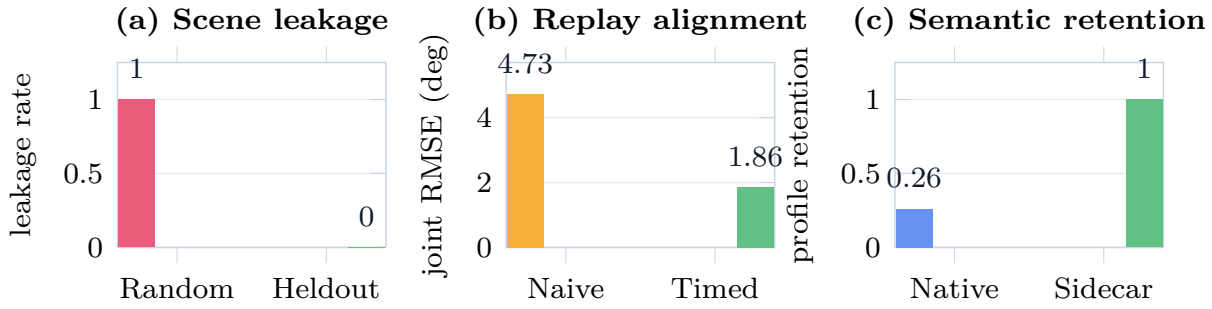

Figure 3: Quantitative summary of the three measured effects used in the evaluation. Lower is better for leakage and replay RMSE; higher is better for semantic retention.

data and data shards, builds a schema-valid WorldEpisode manifest and sidecar, exports a compact LeRobotDataset v3 package, reads it back, and asserts exact equality.

This is a two-dataset batch audit for active conversion, not a coverage claim for all LeRobot datasets. Across ten episodes, the round trip preserves 1,935 rows of source-native action tensors, 1,935 state rows, sample timestamps, frame indices, episode indices, global sample indices, task indices, and camera video timestamp ranges with maximum absolute error 0.0. It also preserves the derived physical-frame records through the sidecar. More importantly, it does not fake missing semantics: camera extrinsics, robot/world calibration, action units, and controller latency are reported as source-absent rather than guessed. This is the difference between a convenient conversion and an auditable one. HELP: C7

Takeaway. *The round trip loses nothing it can preserve (maximum error 0.0) and explicitly names the fields it cannot recover, instead of silently inventing them.*

Case 3: action-timing and replay drift (`ACTION.002`). LeRobot's control stack can decouple policy inference from the fixed-rate low-level controller: commands may be queued, chunked, interpolated, or consumed after policy output [3]. On the public SO-101 trajectory, WorldEpisode records policy-output, enqueue, queue-consume, and motor-receive timestamp semantics, plus zero-order-hold interpolation and a constant-frame latency model.

Estimating the effective delay on the calibration prefix gives four 30 Hz frames, or 133 ms. On the validation suffix, naive command-time alignment has joint RMSE 4.732 degrees. Timestamp-aware alignment has RMSE 1.862 degrees, a 2.54× improvement. We then execute the same WorldEpisode action contract through two minimal six-joint position-servo replay adapters. In MuJoCo, naive command-time replay has RMSE 3.425 degrees and timestamp-aware replay has 1.563 degrees, a 2.19× improvement. Genesis same-trace replay gives the same measured profile: 3.425 degrees under naive command-time replay and 1.563 degrees under timestamp-aware replay, also a 2.19× improvement. The two engines agree to roughly seven significant figures (MuJoCo 3.4247237 versus Genesis 3.4247236 naive) precisely because both integrate the same minimal six-joint position-servo model; we therefore report this as an adapter-conformance result — the same action contract executes consistently across runtimes — and explicitly not as evidence that the engines' underlying physics agree. The Isaac adapter is emitted as a ready mapping from the same contract, but is not tested in this environment. The failure avoided here is a common one: blaming a simulator or controller when the dataset never said when the command actually took effect.

We also add a dependency-free replay-adapter conformance harness before trusting additional runtime adapters. It tests three scheduler cases: constant frame delay, zero-order hold for missing

commands, and asynchronous queue selection. In all three cases a naive scheduler fails while the contract-aware scheduler matches the expected trace with zero numerical error. This is a precondition check for simulator adapters, not a claim that the simulators have equivalent physics. For broader runtime evidence, we also record URDF Studio's shared `SimBackend` conformance suite for MuJoCo and Genesis and its one-episode carton-sorting MuJoCo–Genesis comparison report with matched task success and measured trajectory divergence. These results close the narrow same-trace second-runtime replay gate for the minimal joint replay profile. They do not claim contact-rich task replay, Isaac/SAPIEN execution, or simulator-independent behavior. HELP: C5

Takeaway. *Declaring when a command actually takes effect cuts replay error roughly in half; the drift was a missing timing contract, not a bad simulator.*

Case 4: real-to-sim contract drift (`ACTION.001`, `REP.001`). Recent Gaussian/OpenUSD real-to-sim systems correctly separate visual reconstruction from interactive physical assets. WorldEpisode targets the contract between those layers and the source robot episode. We therefore add two deterministic ablations. In the action-contract ablation, a policy succeeds in a simulator that interprets its output as absolute joint-radian targets, but the same vector fails under a deployment proxy whose controller expects delta-degree commands. In the representation-role ablation, an appearance-only Gaussian export drops the collision proxy; the straight-line policy succeeds in simulation and collides with the real foreground geometry under the deployment proxy. In both ablations the drifted contract has 2/2 simulated successes and 0/2 deployment successes, while the WorldEpisode contract recovers 2/2 deployment successes. This is a controlled proxy, not a RoboSnap or hardware rerun, but it shows why visual reconstruction is not enough without action contracts and representation roles.

Takeaway. *A visually faithful reconstruction still fails at deployment without the action contract and collision role, so appearance alone does not make a scene replay-safe.*

Case 5: validator fault detection (all requirement IDs). We inject 14 faults into the baseline package: missing clock domains, incomplete cross-clock mappings, unknown frames, missing transform intervals, unknown event entities, missing representation roles, digest errors, action-contract omissions, non-content-addressed worlds, missing delta sequences, missing provenance, and missing quality records. The validator detects all 14 expected requirement failures with 1.000 recall and 0.933 precision. Two hand-authored independent fixtures also trigger the expected diagnostics. The single false-positive requirement is useful rather than harmful: removing an action reference frame triggers both `ACTION.001` and the downstream `FRAME.001` reference check.

The same suite includes a simple unit sanity check from `lerobot/svla_so101_pickplace`. The first mirrored SO-101 frame has six joint values with maximum magnitude 98.924 in degrees and 1.727 after radian conversion. Treating those degree values as radians would violate the action/state unit contract even though the tensor shape is valid.

To move beyond synthetic mutations, the experiment runner also writes a pilot natural-source corpus from committed public LeRobot artifacts and selected source-level public benchmark metadata. Across `lerobot/svla_so101_pickplace`, `lerobot/pusht`, `armnet/armnetbench_v01_lerobot_so101`, DROID, and BridgeData V2, it records 19 observed cases. The active LeRobot evidence covers source-absent camera extrinsics, robot/world calibration transforms, action units, controller-latency models, and a random episode split with measured scene-lineage leakage. The DROID and BridgeData V2 cases are source-level public evidence gaps selected from the famous-benchmark call-out audit. These are not claimed as maintainer-confirmed dataset bugs or measured score-inflation results; they are naturally occurring source omissions, metadata

gaps, or evaluation risks that a WorldEpisode audit must make explicit. The runner also emits dataset-specific diagnostic reports for all five datasets; the DROID and BridgeData V2 reports are explicitly marked source-level-only until pinned WorldEpisode conversions and false-positive review are available. HELP: C3

The same validator is exposed as a pre-training command rather than only as an experiment script. The preflight regression covers four cases: a valid WorldEpisode manifest passes, an invalid WorldEpisode fixture fails, a native LeRobot v3 folder without a WorldEpisode sidecar fails closed, and a Rerun `.rrd` recording without a sidecar fails closed. This is the intended usage pattern for labs: run one blocking line before launching a multi-day policy training job. To reduce single-implementation risk, we also run an internal clean-room reader that does not import the `worldepisode` package. It parses the public schema, summarizes the minimal example, and catches all 18 expected requirement instances across 17 pilot and independently authored fixtures with 1.000 recall. This is not external adoption, but it verifies that the public artifacts are readable outside the reference SDK path.

Takeaway. *Every injected physical-coherence fault is caught with full recall, and the same validator runs as a single blocking line before a training job.*

Case 6: dataset-scale manifests and indexes (`DATASET.001`). To address production scale, we audit the dataset manifest in `examples/scalable-corpus.worldepisode-dataset.json`. The audit validates five namespaces, three resolver schemes, three registries, three shards, two indexes, one append-only version, and nine digest-addressed asset descriptors spanning Hugging Face, S3, and OCI URIs with local mirrors. It also checks that the manifest includes a world-lineage index, an asset-digest index, and a split-manifest shard. We then run a generated catalog benchmark with 32,768 trace shards describing 1,073,741,824 episodes. The committed report records the measured time for parsing and indexing the generated JSON catalog, eight partition-pruning queries, digest-cache resolution, and resolver routing over all asset descriptors. This is catalog-side evidence only, but it makes the scale claim executable: opening a corpus starts from the manifest and indexes rather than from recursive storage discovery.

Takeaway. *A billion-episode-capacity corpus opens from its manifest and indexes in milliseconds, without a recursive storage scan.*

Case 7: source-level audit of famous benchmarks (`SPLIT.001`). The ArmnetBench result is the measured leakage case. To scale the same diagnostic, we add a source-level call-out audit over five famous public robot-learning benchmarks: Open X-Embodiment/RT-X [17], DROID [5], BridgeData V2 [22], LIBERO [12], and CALVIN [4]. The audit asks whether public metadata exposes world or scene lineage splits, content-addressed world revisions, persistent entity identity, action timing and latency contracts, frame/clock graphs, and replay or conversion-loss reports. All five benchmarks have at least one high-severity open control; the three real-robot datasets have five each in the current source-level audit. This is a call-out target list, not a claim that their published scores are inflated. The stronger claim requires converting each benchmark into WorldEpisode and rerunning a published policy protocol under lineage-disjoint splits or timestamp-aware replay. To make that boundary executable, we add a separate inflation-proof gate: it requires a benchmark-specific WorldEpisode conversion, lineage/timing audit, published-policy rerun, corrected evaluation, and measured score drop before any famous benchmark can be labeled inflated. We also add a targeted DROID-100 LeRobot-subset rerun tool that downloads pinned public Parquet shards, builds a WorldEpisode evidence package, compares random and lineage-disjoint splits, and runs a deterministic offline state-action policy. In the current run, that DROID subset tool executes over

100 public episodes and 32,212 frames. Its lineage source is only a task/camera-schema proxy, not a physical scene identifier; the corrected proxy split scores 0.516 versus 0.496 for the random split, so the measured score drop is negative. The proof gate therefore records one executed DROID subset rerun, zero inflation-proof famous-benchmark rerun reports, and zero measured famous-benchmark inflation claims in this release.

Open result, not claimed: famous-benchmark score-inflation proof

Status: open. A targeted DROID subset rerun can be attempted with `uv run -with pyarrow -with requests -with numpy python tools/famous_benchmark_policy_rerun.py -benchmark droid_100 -required`. The proof contract is then enforced with `python3 tools/benchmark_inflation_gate.py -required`.

Required artifacts: benchmark-specific WorldEpisode conversion, lineage/timing audit, faithful published-protocol policy rerun, paired corrected evaluation under the same metric, and a measured baseline-minus-corrected score drop.

Acceptance rule: the gate must contain at least one inflation-proof valid rerun report with `measured_inflation=true`. Source-level metadata gaps alone are evidence of missing controls, not evidence that published scores are inflated.

HELP: C2

Takeaway. *The famous benchmarks lack the public controls needed to rule out leakage; we flag that gap but refuse to call any score inflated without a measured rerun.*

Case 8: world-grounded counterfactual augmentation (`ENTITY.001`). We evaluate a shifted-camera/object-displacement regression task. Observations-only nearest-neighbor control reaches the proxy success threshold on 0.292 of shifted test cases. Adding unstructured 3D side information increases proxy success to 0.342. WorldEpisode-style world-frame identity and counterfactual camera augmentation increase proxy success to 0.592 over 120 deterministic test cases.

Takeaway. *Grounding augmentation in world-frame identity beats both raw observations and unstructured 3D side files on shifted-camera tests.*

Binding retention. The same executable artifacts measure how much of the semantic contract is preserved by native containers alone. Each binding writes a native payload and a WorldEpisode sidecar, then the experiment reimports both and checks exact equality against the versioned `uss-core-23` projection in `conformance/projections/uss-core-23.v0.json`: episode identity/task outcome, world revision and asset descriptor, embodiment identity and URDF [16] asset, frames/transforms, clocks/mappings, entity identity/roles/assets, action control and timing contracts, trace binding and asset descriptor, events, ordered deltas, provenance, quality, split lineage, and replay assumptions. This is an executable pilot projection, not a universal score of each format's value. Across seven pilot bindings, Table 2 shows that common containers preserve 17–39% of this projection natively, while WorldEpisode sidecars recover that projection for the tested dataset/log/world bindings. The sidecar column is by construction: because the sidecar serializes the projection fields directly, the 1.000 entries verify lossless round-trip of the contract, not an emergent property; the informative measurement is the native gap each container leaves. A Gaussian asset binding alone preserves 17% natively and 30% with limited sidecar metadata, confirming that an asset format is not an episode–world contract.

Takeaway. *Native containers preserve only 17–39% of the semantic contract and the sidecar restores it, so an asset or log format alone is not an episode–world contract.*

Table 2: Semantic field retention in the pilot binding model.

Binding	Native	With WorldEpisode
LeRobot v3	0.261	1.000
Rerun RRD	0.348	1.000
NCore	0.304	1.000
MCAP/ROS2	0.261	1.000
OpenUSD/SimReady	0.391	1.000
glTF Gaussian asset	0.174	0.304

Finally, we generate episodes with world/entity lineage labels and evaluate a memorization policy. A random episode split leaks every tested world/entity pair and obtains 1.000 accuracy. World-disjoint and entity-disjoint splits have zero measured leakage and reduce memorization accuracy to 0.500 and 0.562 respectively. This pilot demonstrates why split manifests need lineage constraints beyond the public ArmnetBench audit above.

7 Limitations and Open Gates

What the evidence supports, and, explicitly, what it does not yet support.

The current evidence is deliberately scoped. It is meant to expose hidden failure mechanisms, not to claim full robot-learning, game-engine, or autonomous-driving generality.

Table 3 states the boundary of each empirical claim. This is important because the paper argues for a semantic contract, not for a finished benchmark suite.

Table 3: What the current evidence supports, and what it does not yet support.

Claim area	Current evidence	Boundary
Leakage	Public ArmnetBench LeRobot audit with 400 teleoperated reference episodes, an executable Torch BC probe, a measured temporal ridge state/action baseline, an ACT/Diffusion gate harness, and four compact digest-verified LeRobot state/action split packages totaling 241,470 frames	Stronger ACT/Diffusion jobs are prepared but not run; the compact split packages omit video payloads, so no vision-policy, real-robot, or high-fidelity rollout result is claimed yet
Conversion	Two pinned public LeRobotDataset v3 five-episode batch round trips with exact tensor, index, and timestamp equality	Two datasets; broader LeRobot coverage remains future work
Replay timing	Real SO-101 trajectory alignment, tested same-trace MuJoCo and Genesis position-servo replay adapters, URDF Studio MuJoCo/Genesis episode-backend evidence, and a dependency-free scheduler conformance harness	One LeRobot trace with minimal joint replay adapters; no contact-rich task rollout, Isaac runtime result, or SAPIEN runtime result is claimed
Meta-simulator neutrality	Adapter contract matrix over MuJoCo, Isaac Sim, Genesis, and SAPIEN plus same-trace MuJoCo/Genesis replay evidence, URDF Studio `SimBackend` conformance, and a one-episode MuJoCo–Genesis comparison	MuJoCo and Genesis are tested for the minimal replay profile; Isaac and SAPIEN are not replay-tested in this paper, and equal physics is not claimed
Generalization beyond robotics	Deterministic game-engine collision-patch and autonomous-driving clock-domain pilots using the same invariant vocabulary	Not measured on production game engines, public AV datasets, or fleet logs
Validation	Fourteen injected requirement faults, two independent hand-authored fixtures, and a pilot natural-source corpus over five public datasets	Five-dataset count is met through active LeRobot artifacts plus source-level public benchmark metadata; dataset-specific diagnostic reports now cover all cases, but no maintainer feedback is recorded yet
Binding retention	Versioned `uss-core-23` semantic projection checked by executable artifacts	Pilot projection; not a universal score of each storage format
Famous benchmark call-out	Source-level audit over Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN, plus a targeted DROID subset rerun tool and a separate inflation-proof gate	Missing public controls are not measured score inflation; the proof gate currently has one executed DROID subset rerun, zero inflation-proof famous-benchmark rerun reports, and zero measured inflation claims
Real-to-sim drift	Controlled action-contract and representation-role ablations where drifted contracts succeed in simulation and fail under deployment proxies	Deterministic proxy; not a RoboSnap/DROID-Sim rerun or physical hardware deployment
Adoption	Public schema, installable local preflight package, validator, fixtures, governance files, and an internal clean-room reader that does not import the reference SDK	No PyPI release, upstream LeRobot/Rerun integration, external independent implementation, or external dataset release yet
Scale architecture	Dataset manifest schema, scalable example, executable audit, and a generated 32,768-shard catalog benchmark describing 1,073,741,824 episodes for open/index latency, partition pruning, digest-cache behavior, and resolver routing	Catalog-side benchmark only; no billion episode rows, payload bytes, distributed storage, concurrent readers, or institutional federation are measured

Open result, not claimed: results not claimed in this release

The artifact keeps unfinished claims executable instead of hiding them. The current RFC release gate is: `python3 tools/release_readiness.py -strict-rfc`. The full-standard gate is: `python3 tools/release_readiness.py -strict-full`. The latter is expected to fail until all open gates below are backed by committed reports. The same gates are indexed in `docs/experiments/open_reproduction_gates/open_reproduction_gates.json`.

- ACT/Diffusion leakage: run `python3 tools/lerobot_policy_leakage_gate.py`, then execute `docs/experiments/lerobot_policy_gate/run_lerobot_policy_jobs.sh`. A paper update must commit the policy metrics, rollout traces or videos, and updated `docs/experiments/lerobot_policy_gate/policy_gate_report.json`.
- Famous-benchmark inflation: run the targeted DROID subset rerun with optional Arrow/HTTP dependencies, using `tools/famous_benchmark_policy_rerun.py` with `-benchmark droid_100 -required`, then require `python3 tools/benchmark_inflation_gate.py -required`. No benchmark may be described as inflated unless this gate records an inflation-proof valid rerun and a measured corrected-score drop.
- External standard evidence: add maintainer-reviewed diagnostics, an independently written reader/exporter, and at least one externally published WorldEpisode-compatible dataset before calling the project a mature interoperability standard.

Beyond robotics (future work). All measured evidence in this paper is the WorldEpisode robotics profile; the broader Universal Spatial State framing is proposed, not demonstrated outside robotics. Two deterministic pilots only illustrate that the same vocabulary can express non-robotics drift. In a game-engine collision pilot, a client loads a stale arena collision asset after an authoritative patch, which the asset-digest, state-revision, and role checks catch. In an autonomous-driving clock pilot, camera and lidar logs deserialize correctly, but a 50 ms undeclared clock offset at 15 m/s creates a 0.75 m fusion error against a 0.20 m tolerance, which the declared clock mapping corrects to 0.0 m. These are illustrative pilots, not production game-engine or AV benchmark results. A stronger paper should add at least one public non-robotics corpus and an independently maintained adapter. HELP: C6

Offline leakage result. The ArmnetBench experiment is an offline behavioral-cloning audit, not a physical rollout. Its value is that it demonstrates how random episode splits can leak world lineage and inflate evaluation, not that it measures real-robot success directly. The current baseline is an executable Torch MLP probe over LeRobot tensors; ACT and Diffusion Policy should be run on the committed split manifest before claiming impact on common LeRobot training recipes. The repository now also measures a temporal ridge baseline over the committed compact packages; it drops from 0.925 random episode offline success to 0.420 scene-disjoint success, but remains a low-dimensional offline state/action result. The repository emits ACT/Diffusion jobs, compact state/action LeRobot split packages, and rollout requirements, but no ACT/Diffusion policy result is claimed here. The split packages verify 27 cached source files, preserve state/action/timestamp rows, and remap episode/index fields into contiguous local LeRobot folders; they intentionally omit video payloads, so a vision-policy result still requires mirrored source videos with committed digests.

Meta-simulator evidence. The meta-simulator section (Section 4) defines what a compliant runtime adapter must preserve; it does not certify that MuJoCo, Isaac, Genesis, SAPIEN, or any other simulator has equivalent physics, and neither the replay results nor the replay-adapter harness are a claim of simulator-independent behavior. Isaac and SAPIEN remain untested here, and no contact-rich task rollout result is claimed.

Reference implementation maturity. The converter, validator, and reports are executable, but independent implementations are still needed before the project should be treated as a mature interoperability standard. The clean-room reader reduces the risk that the fixtures only work through the reference SDK, but it is still an internal artifact, and the `worldepisode` preflight CLI and API are not yet a released PyPI package or merged into upstream LeRobot or Rerun examples. The next empirical step is to run the same protocol on larger multi-camera manipulation datasets and externally maintained adapters. HELP: C4

Famous benchmark call-out scope. The audit over Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN is a source-level metadata audit; it does not prove any published benchmark score is inflated without a benchmark-specific conversion and a measured policy rerun under the corrected split or timing contract.

Natural failure corpus. The injected conformance faults prove that requirements are executable and diagnosable. The current pilot natural-source corpus adds 19 cases from five public robot-learning datasets, but it still does not prove prevalence in the wild. A stronger version of this paper should convert the source-level gaps into dataset-specific WorldEpisode manifests, report false-positive review, and record whether dataset maintainers agree with representative diagnostics.

8 Positioning

Where WorldEpisode sits relative to existing formats, simulators, benchmarks, and standards bodies.

8.1 Related work

Documenting datasets is not new. Datasheets for Datasets [6] and model cards standardize prose descriptions of provenance and intended use; MLCommons Croissant [13] adds a machine-readable metadata layer for ML-ready datasets; and Open X-Embodiment [17] aggregates robot data under a shared schema. Evaluation leakage is a well-documented driver of the reproducibility crisis in ML-based science [9]. WorldEpisode differs in three ways. First, it binds these fields to an *immutable, content-addressed world revision* with closed identity, frame/clock, role, and provenance graphs, so cross-container agreement is checkable rather than merely described. Second, it is *executable and fail-closed*: a one-line preflight blocks a native LeRobot or Rerun artifact that cannot prove the contract, and every conversion emits a machine-readable loss report. Third, it treats *lineage-disjoint evaluation splits* as a first-class requirement with measured consequences, not as optional documentation. It is a contract and validator layered on existing containers, not a replacement format or a dataset card.

A contribution of falsifiable invariants. The experiments show several ways a result can be wrong while the files are valid: a split can leak world lineage, replay can drift because control timing is underspecified, conversion can discard physical semantics that the target container has no native place to store, and a live virtual or AV state can drift from its authoritative revision. WorldEpisode is useful only because these failures are observable and testable. The sidecar is the mechanism; the scientific claim is that explicit spatial-state semantics change what an embodied-agent result means. The contribution is therefore a set of falsifiable invariants: persistent entity identity must be consistent across observations and assets, timestamps must distinguish observation, command, and effective time, state roles must separate appearance from physics, and evaluation splits must be able

to exclude shared state lineage. A container adapter either preserves those invariants, externalizes them with declared loss, or fails conformance.

A small semantic core with multiple bindings. WorldEpisode has a small semantic core and multiple bindings: LeRobotDataset v3 is the policy-training view; Rerun is the multirate physical-data view; NCore is the sensor/reconstruction capture view; MCAP/ROS is the raw log view; OpenUSD/SimReady is the composed simulation-world view; glTF or SPZ is the Gaussian asset view; and a reference directory is the conformance profile. OpenUSD standardizes how the 3D world is composed; WorldEpisode standardizes how a robot modifies state within that space over time without silent data corruption. The same invariant structure should generalize to a game-engine or AV profile, though only the robotics profile is implemented and evaluated here.

Relationship to real-to-sim pipelines. Gaussian-splat and OpenUSD real-to-sim pipelines are not competitors to WorldEpisode. They are the systems that most need a state and episode contract. A reconstructed room can be excellent visual training data and still be unsafe for policy evaluation if foreground collision roles are missing, if the same world lineage leaks across splits, or if the replayed policy output is interpreted under the wrong hardware action contract. WorldEpisode should be judged by whether it keeps those pipelines auditable as they scale, not by whether it serializes splats better than asset formats designed for that job.

Simulators as accountable adapters. The same principle applies to physics and rendering engines. WorldEpisode should not become a MuJoCo wrapper, an Isaac wrapper, or a Genesis wrapper. It should define the contract those adapters must satisfy: ingest the invariant interface, declare extensions, and report replay assumptions and drift. This is a stronger runtime-neutral stance than claiming universal behavior. It lets a simulator team plug in by implementing the adapter, while the validator remains the common judge of whether the exported dataset is physically and temporally auditable.

Lineage-safe evaluation. Record-level random splits can leak the same reconstructed room, physical object, Gaussian asset, source video, state revision, map patch, or generated asset lineage across training and evaluation. WorldEpisode supports split constraints such as `world_lineage`, `physical_entity`, `source_capture`, `reconstruction_run`, and `generated_asset_family`.

Auditing benchmarks without overclaiming. The practical target is to make famous benchmarks auditable, not to accuse them without a rerun. When public metadata cannot answer whether train and test share a scene lineage, asset family, source capture, action timing convention, or simulator assumption, WorldEpisode should mark the published score as unaudited with respect to that control. It should mark a score as inflated only after the same policy protocol is rerun under the corrected split or timing contract.

Standards alignment. The project aligns with standards bodies rather than antagonizing them. Static 3D robot-map exchange, humanoid dataset lifecycle, OpenLABEL annotation, glTF Gaussian splats, USD composition, and NCore capture conventions are all useful neighbors. WorldEpisode is the executable profile and bridge across those layers.

9 Conclusion

Spatial-agent failures often arrive disguised as clean data: valid tensors, valid videos, valid scene files, valid telemetry logs, and invalid state assumptions. WorldEpisode makes those assumptions explicit. It binds interaction data to immutable state revisions, gives persistent identity to entities, declares space/time/transition semantics, assigns roles to assets, records events and provenance, and reports conversion loss instead of hiding it.

In the reference experiments, those requirements expose an 85% offline random-split result that collapses to 0% under scene-disjoint evaluation, reduce LeRobot replay drift on a real SO-101 trajectory by modeling a four-frame action delay, and catch every injected physically incoherent package before deployment. A real-to-sim pipeline can report simulated success while failing under deployment proxies when action contracts or representation roles drift; the same vocabulary catches that drift in deterministic pilots outside robotics as well. The validator ships as a one-line preflight command, and the simulator pathway is an adapter compliance contract rather than a dependency on one runtime.

These are scoped results, not a completed standardization claim. The call-out audit identifies Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN as priority targets for this protocol, but it does not claim their scores are inflated without reruns; the inflation-proof gate currently records one executed DROID subset rerun, zero inflation-proof famous-benchmark rerun reports, and zero measured inflation claims. The non-robotics pilots are vocabulary checks, not production game-engine or AV evaluations. Gaussian splats remain a high-value appearance profile, but the durable contribution is the spatial-state contract around them. The next step is to scale the same protocol to larger multi-robot datasets, stronger LeRobot-native policies, real or high-fidelity rollout evaluation, natural failure corpora, non-robotics adapters, and independently maintained bindings.

A Open Contributions (colleague working notes)

This appendix is a coordination surface, not a paper result. It renders only in the working build (the `\collabtrue` switch in `main.tex`); flip that switch to `\collabfalse` to remove it before any external submission. Each card below is a claimable unit of work with a fixed anchor: **Where** it lives in this paper and the repo, **What is missing**, **How to close it** (exact commands), and **Needs** (environment and access). The purple HELP: Cx tags in the Failure Case Studies and Limitations sections point back to these cards.

How to claim a card. Put your name in the **Owner** slot, open an RFC issue for any normative change (per `GOVERNANCE.md`), and keep published requirement IDs stable. The four experiment gates (C1–C4) are the same ones indexed in `docs/experiments/open_reproduction_gates/open_reproduction_gates.json`.

Finishing a card also means finishing the prose. The paper text is complete but evidence-gated: closing a card is not done until you (1) fold the measured numbers into the cited section, (2) update the evidence-boundary table in the Limitations section and the summary figure in the Failure Case Studies section, and (3) keep the gates green by re-running `python3 tools/paper_claim_audit.py --strict` and `python3 tools/artifact_freshness.py --strict`. The whole readiness suite is `make readiness`.

Experiment gates

[C1] ACT/Diffusion policy metrics and rollout impact

Where: Evidence: Failure Case Studies, Case 1 (scene-lineage leakage); Limitations, "Offline leakage result"; artifacts under `docs/experiments/lerobot_policy_gate/`. Gate `POLICY.ROLL.001`.

What is missing: ACT and Diffusion Policy offline metrics on *both* the random and scene-disjoint splits, plus at least one simulator or physical rollout on the same split manifest. Source videos must be mirrored with digests before any vision-policy claim; the current gate is `ready_not_executed`.

How to close it: `python3 tools/lerobot_policy_leakage_gate.py`, then `bash docs/experiments/lerobot_policy_gate/run_lerobot_policy_jobs.sh` in a LeRobot training environment. The compact physical split packages under `docs/experiments/lerobot_policy_gate/physical_splits/` are already committed, so offline training needs no download; only source videos must be mirrored (with digests) for a vision policy. Commit the policy metrics, rollout traces or videos with digests, and the updated `docs/experiments/lerobot_policy_gate/policy_gate_report.json`.

Needs: GPU plus a LeRobot / ACT / Diffusion Policy training environment; Hugging Face access to the source videos.

Owner: _____ **Target:** _____

[C2] Famous-benchmark score-inflation proof

Where: Evidence: Failure Case Studies, Case 7 (famous benchmarks); Limitations, "Famous benchmark call-out scope". Gate `BENCH.INFLATE.001`.

What is missing: A benchmark-specific WorldEpisode conversion, a faithful published-protocol policy rerun, and a paired corrected evaluation showing a measured baseline-minus-corrected score drop. The committed DROID-100 attempt is fail-closed (Hugging Face DNS resolution failed) and its task/camera proxy split shows no drop, so zero inflation-proof reports exist yet.

How to close it: `uv run --with pyarrow --with requests --with numpy python tools/famous_benchmark_policy_rerun.py --benchmark droid_100 --required`, then `python3 tools/benchmark_inflation_gate.py --required`. Offline: pre-place the pinned Parquet shards under `--cache-root <dir>` at the repo-id/revision path the tool prints, then re-run with that flag to skip all network access (the runner also attempts a DNS-over-HTTPS fallback automatically).

Needs: Reliable network access to Hugging Face; the published policy checkpoint and evaluation protocol for the target benchmark; compute for the rerun.

Owner: _____ **Target:** _____

[C3] Maintainer-confirmed natural-failure prevalence

Where: Evidence: Failure Case Studies, Case 5 (the validator) and its natural-source corpus; Limitations, "Natural failure corpus". Gate: natural-failure prevalence.

What is missing: Pinned WorldEpisode conversions for the source-level datasets (DROID, BridgeData V2), false-positive review of the emitted diagnostics, and recorded agreement from the dataset maintainers. Today two cases are source-level metadata only.

How to close it: Extend the converters to pin those datasets, re-run `python3 tools/run_experiments.py`, then contact the dataset maintainers with representative diagnostics and record their responses.

Needs: Dataset domain knowledge; maintainer contacts; storage for the pinned shards.

Owner: _____ **Target:** _____

[C4] Mature external-standard adoption

Where: Limitations, "Reference implementation maturity" and the Adoption row of the evidence-boundary table; `GOVERNANCE.md` "Independence Target". Gate: external adoption.

What is missing: A second, independently written reader/exporter (not the reference SDK), at least one externally published WorldEpisode-compatible dataset, a PyPI release of the `worldepisode` package, and upstream LeRobot / Rerun integration.

How to close it: Build the independent implementation and validate it with `python3 tools/cleanroom_conformance_reader.py`; publish the wheel to PyPI; open the upstream pull requests; re-check `python3 tools/release_readiness.py --strict-rfc`.

Needs: A second implementer, ideally at another institution (the independence requirement); packaging and release access.

Owner: _____ **Target:** _____

Standalone technical items

[C5] Isaac Sim / SAPIEN replay and a contact-rich rollout

Where: Evidence: Failure Case Studies, Case 3 (the replay that drifted); Limitations, "Meta-simulator evidence".

What is missing: Tested Isaac Sim and SAPIEN replay adapters (Isaac is adapter-ready but untested; SAPIEN is adapter-required) and at least one contact-rich task rollout. Only MuJoCo and Genesis same-trace replay is measured today.

How to close it: Implement the adapters against the meta-simulator contract, validate scheduling with `python3 tools/replay_adapter_conformance.py`, then run real Isaac / SAPIEN same-trace replays and commit the RMSE reports.

Needs: IsaacLab and SAPIEN installations; GPU.

Owner: _____ **Target:** _____

[C6] A public non-robotics corpus (generalization beyond robotics)

Where: Limitations, "Beyond robotics (future work)".

What is missing: At least one public non-robotics corpus (game-engine or autonomous-driving) bound through an independently maintained adapter, beyond the two deterministic pilots.

How to close it: Pick a public AV or game-telemetry dataset, write a binding plus a conversion-loss report, add conformance fixtures, and wire it into `tools/run_experiments.py`.

Needs: Access to a public AV or game-telemetry dataset; a maintainer for that adapter.

Owner: _____ **Target:** _____

[C7] Broader conversion coverage

Where: Evidence: Failure Case Studies, Case 2 (the converter that refuses to invent physics); Limitations, Conversion row of the evidence-boundary table.

What is missing: More than two round-tripped LeRobot datasets, and at least one larger multi-camera manipulation dataset.

How to close it: `python3 tools/lerobot_worldepisode_roundtrip.py --required --repo-id <dataset> --revision <sha> --batch-episode-indices 0,1,2,3,4`; commit the round-trip reports.

Needs: Hugging Face access; disk for the shards.

Owner: _____ **Target:** _____

References

[1] Alliance for OpenUSD. OpenUSD: Universal scene description. https://openusd.org/, 2026. Accessed 2026-07-13.

[2] Armnet. ArmnetBench v0.1: LeRobot single-arm SO-101 release. https://huggingface.co/datasets/armnet/armnetbench_v01_lerobot_so101, 2026. Pinned revision 2e5e89aee0e7f081078d9d6ab3b279fc4b83ea84; accessed 2026-07-13.

[3] Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, Mustafa Shukor, Jess Moss, Alexander Soare, Dana Aubakirova, Quentin Lhoest, Quentin Gallouedec, and Thomas Wolf. LeRobot: An open-source library for end-to-end robot learning. In *International Conference on Learning Representations*, 2026. arXiv:2602.22818.

[4] CALVIN Authors. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. https://calvin.cs.uni-freiburg.de/, 2021. Accessed 2026-07-13.

[5] DROID Dataset Authors. DROID: A large-scale in-the-wild robot manipulation dataset. https://droid-dataset.github.io/, 2024. Accessed 2026-07-13.

[6] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

[7] Google DeepMind. MuJoCo: Multi-joint dynamics with contact. https://mujoco.org/, 2026. Accessed 2026-07-13.

[8] Hugging Face. LeRobotDataset v3.0. https://huggingface.co/docs/lerobot/lerobot-dataset-v3, 2026. Accessed 2026-07-13.

[9] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.

[10] LeRobot. pusht: A public LeRobotDataset v3 dataset. https://huggingface.co/datasets/lerobot/pusht, 2025. Pinned revision 7628202a2180972f291ba1bc6723834921e72c19; accessed 2026-07-13.

[11] LeRobot. svla_so101_pickplace: A public LeRobotDataset v3 dataset. https://huggingface.co/datasets/lerobot/svla_so101_pickplace, 2025. Pinned revision f641879e22172be7e8161d5e6c1503c2d2feb657; accessed 2026-07-13.

[12] LIBERO Authors. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. https://libero-project.github.io/main.html, 2023. Accessed 2026-07-13.

[13] MLCommons. Croissant: A metadata format for ML-ready datasets. https://mlcommons.org/croissant/, 2024. Accessed 2026-07-13.

[14] NVIDIA. NCore Specification: Data Conventions. https://nvidia.github.io/ncore/data/conventions, 2026. Accessed 2026-07-13.

[15] Open Robotics. ROS REP 103: Standard units of measure and coordinate conventions. https://www.ros.org/reps/rep-0103.html, 2026. Accessed 2026-07-13.

[16] Open Robotics. URDF: Unified robot description format. https://wiki.ros.org/urdf, 2026. Accessed 2026-07-13.

[17] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and rt-x models. https://robotics-transformer-x.github.io/, 2023. Accessed 2026-07-13.

[18] Rerun. A new data layer for robot learning. https://rerun.io/blog/data-layer-for-robot-learning, 2026. Accessed 2026-07-13.

[19] RoboSnap Authors. RoboSnap: One-Shot Real-to-Sim Scene Generation for Generalizable Robot Learning and Evaluation. https://arxiv.org/html/2607.06699v1, 2026. Accessed 2026-07-13.

[20] The Khronos Group. KHR_gaussian_splatting. https://github.com/KhronosGroup/glTF/blob/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md, 2026. Accessed 2026-07-13.

[21] The Khronos Group. Khronos Announces glTF Gaussian Splatting Extension. https://www.khronos.org/news/press/gltf-gaussian-splatting-press-release, 2026. Accessed 2026-07-13.

[22] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Quan Vuong, Philippe Hansen-Estruch, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Conference on Robot Learning*, 2023. Accessed 2026-07-13.